

DOCUMENTATION TECHNIQUE COMPLETE

Docker, Docker Compose et Jenkins

Portfolio React + Express + MongoDB

Explication ordonnee, detaillee et ligne par ligne des Dockerfiles, du fichier Compose, de Nginx, des ports, de `npm ci`, de la base MongoDB, du dossier `dist` et du pipeline Jenkins.

Projet	REACTPORTFOLIO
Frontend	React + Vite servi par Nginx
Backend	Express.js + Mongoose
Base de donnees	MongoDB via image officielle <code>mongo:7</code>
CI/CD	Jenkins avec build, tests, images Docker, deploiement et health check

Table des matieres

1. Vue d'ensemble du projet
2. Definitions Docker essentielles
3. Relation Dockerfile, image, conteneur et Docker Hub
4. Explication de `npm ci`
5. Explication des ports
6. Dockerfile backend Express
7. Dockerfile frontend React
8. Le dossier `dist`
9. Fichier `compose.yaml`
10. MongoDB, image Docker Hub et volume
11. Configuration Nginx
12. Jenkins avec Docker
13. Jenkinsfile ligne par ligne
14. Health checks et correction de l'erreur du pipeline
15. Commandes utiles

1. Vue d'ensemble du projet

Ton projet est une application en trois parties. Le frontend React affiche l'interface utilisateur. Le backend Express expose une API REST. MongoDB stocke les données des projets du portfolio.

Navigateur

- > `http://localhost:3000`
- > conteneur frontend Nginx, port 80
- > fichiers React compiles dans `/usr/share/nginx/html`
- > appels `/api` envoyés au backend
- > conteneur backend Express, port 5000
- > connexion MongoDB
- > conteneur mongo, port 27017, volume mongo-data

Les fichiers principaux sont :

Fichier	Role
<code>reactportfolio/Dockerfile</code>	Construit le frontend React avec Node, puis sert le resultat avec Nginx.
<code>EXPRESSJS</code> <code>PORTFOLIO/Dockerfile</code>	Construit l'image du backend Express.
<code>compose.yaml</code>	Lance ensemble frontend, backend et MongoDB.
<code>reactportfolio/nginx.conf</code>	Configure Nginx pour servir React et relayer les appels API.
<code>Jenkinsfile</code>	Definit le pipeline CI/CD : checkout, install, tests, build, Docker, deploy, verification.
<code>Dockerfile.jenkins</code>	Construit une image Jenkins personnalisee avec Node.js et Docker CLI.
<code>docker-compose.jenkins.yaml</code>	Lance Jenkins dans un conteneur avec acces au Docker de la machine.

2. Definitions Docker essentielles

Docker

Docker est une plateforme qui permet d'emballer une application avec tout ce dont elle a besoin pour fonctionner : systeme minimal, dependances, fichiers, variables et commande de demarrage.

Dockerfile

Un Dockerfile est une recette. Il explique comment construire une image. Il contient des instructions comme `FROM`, `WORKDIR`, `COPY`, `RUN`, `ENV`, `EXPOSE` et `CMD`.

Image Docker

Une image est le resultat construit a partir d'un Dockerfile ou telecharge depuis un registre comme Docker Hub. Elle est immuable : elle sert de modele.

Conteneur

Un conteneur est une image en cours d'execution. Si l'image est le modele, le conteneur est l'instance vivante.

Docker Compose

Docker Compose permet de declarer plusieurs conteneurs dans un seul fichier YAML. Dans ton projet, Compose lance le frontend, le backend et MongoDB, puis les place sur le meme reseau.

Docker Hub

Docker Hub est un registre public d'images. Par exemple, `mongo:7`, `node:20-slim`, `node:20-alpine` et `nginx:alpine` viennent de Docker Hub si elles ne sont pas deja presentes localement.

3. Relation Dockerfile, image, conteneur et Docker Hub

Dockerfile local

- > docker build
- > image personnalisee
- > docker run
- > conteneur en execution

Pour obtenir un conteneur, il faut toujours une image. Mais pour obtenir une image, il y a deux possibilites :

Cas	Exemple	Explication
Image construite localement	<code>build:</code> <code>./reactportfolio</code>	Docker lit le Dockerfile du dossier et construit une image frontend.
Image recuperee depuis Docker Hub	<code>image: mongo:7</code>	Docker telecharge l'image officielle MongoDB. Tu n'as pas besoin d'ecrire un Dockerfile MongoDB.

La base de donnees a bien une image. Elle n'a simplement pas de Dockerfile dans ton projet parce que l'image officielle `mongo:7` existe deja.

4. Explication de npm ci

Dans les deux Dockerfiles, on trouve :

```
COPY package.json package-lock.json ./
RUN npm ci
```

`npm ci` installe les dependances Node.js de façon propre, stricte et reproductible. Il lit `package-lock.json` et installe exactement les versions indiquées.

Element	Utilite
<code>package.json</code>	Liste les dependances directes et les scripts du projet.
<code>package-lock.json</code>	Verrouille les versions exactes de toutes les dependances.
<code>npm ci</code>	Installe exactement ce qui est dans le lock file, sans le modifier.
<code>node_modules</code>	Dossier genere contenant les dependances installees.

Difference entre npm install et npm ci

Commande	Comportement	Quand l'utiliser
<code>npm install</code>	Installe les dependances et peut modifier <code>package-lock.json</code> .	Pratique en developpement quand tu ajoutes ou modifies des dependances.
<code>npm ci</code>	Supprime <code>node_modules</code> s'il existe, puis reinstalle exactement les versions du lock file.	Ideal pour Docker, Jenkins et les builds automatiques.

Dans Docker, on prefere `npm ci` car l'objectif est de construire la meme image sur toutes les machines. Cela evite les surprises entre ton PC, Jenkins et un serveur de production.

5. Explication des ports

Un port est une porte reseau. Une application ecoute sur un port pour recevoir des connexions. Dans Docker, il faut distinguer le port de la machine et le port du conteneur.

```
ports:
- "PORT_MACHINE:PORT_CONTENEUR"
```

Service	Port machine	Port conteneur	Utilite
Frontend	3000	80	Le navigateur ouvre <code>http://localhost:3000</code> , Docker redirige vers Nginx.
Backend	5000	5000	L'API Express est accessible sur <code>http://localhost:5000</code> .
MongoDB	27017	27017	MongoDB Compass peut se connecter a <code>mongodb://localhost:27017</code> .
Jenkins	8080	8080	Interface web Jenkins.
Agents Jenkins	50000	50000	Communication entre Jenkins et ses agents.

Regle importante : depuis ta machine, on utilise `localhost` . Entre conteneurs, on utilise les noms Docker : `portfolio-backend` , `backend` , `mongo` , selon la configuration du reseau.

Depuis ta machine :

```
localhost:3000 -> frontend:80
localhost:5000 -> backend:5000
localhost:27017 -> mongo:27017
```

Depuis les conteneurs :

```
frontend -> portfolio-backend:5000
backend -> mongo:27017
```

6. Dockerfile backend Express

```
FROM node:20-slim

WORKDIR /app

COPY package.json package-lock.json ./

RUN npm ci

COPY app.js ./
COPY src/ ./src/

ENV USE_MEMORY_DB=true
ENV PORT=5000

EXPOSE 5000

CMD ["node", "app.js"]
```

Ligne	Explication
FROM node:20-slim	Point de depart de l'image. Elle contient Node.js 20 dans une version Linux allgee. Le backend a besoin de Node pour executer <code>app.js</code> .
WORKDIR /app	Cree et utilise le dossier <code>/app</code> dans le conteneur. Les commandes suivantes s'executent dans ce dossier.
COPY package.json package-lock.json ./	Copie les fichiers de dependances avant le code. Docker peut ainsi reutiliser le cache si le code change mais pas les dependances.
RUN npm ci	Installe les dependances backend : Express, CORS, Mongoose, dotenv, mongodb-memory-server, etc.
COPY app.js ./	Copie le point d'entree de l'API Express.
COPY src/ ./src/	Copie le dossier contenant les routes et le modele MongoDB.
ENV USE_MEMORY_DB=true	Valeur par defaut : utiliser MongoDB en memoire. Dans Compose et Jenkins, cette valeur est remplacee par <code>false</code> .
ENV PORT=5000	Definit le port par defaut de l'application Express.
EXPOSE 5000	Documente que le conteneur ecoute sur le port 5000. Ne publie pas le port tout seul.
CMD ["node", "app.js"]	Commande lancee quand le conteneur demarre. Si cette commande s'arrete, le conteneur s'arrete.

Lien avec app.js

```
const PORT = process.env.PORT || 5000
...
const useMemoryDB = process.env.USE_MEMORY_DB === 'true'
...
const mongoUri = process.env.MONGO_URI || 'mongodb://localhost:27017/portfolio'
...
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`)
})
```

Le backend lit les variables d'environnement. Si `USE_MEMORY_DB=true`, il lance une base temporaire avec `mongodb-memory-server`. Si `USE_MEMORY_DB=false`, il utilise `MONGO_URI` pour se connecter au vrai conteneur MongoDB.

7. Dockerfile frontend React

```
FROM node:20-alpine AS build

WORKDIR /app

COPY package.json package-lock.json ./

RUN npm ci

COPY index.html vite.config.js styles.css ./
COPY src/ ./src/
COPY assets/ ./assets/

RUN npm run build

FROM nginx:alpine

COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Pourquoi deux FROM ?

Ce Dockerfile utilise un build multi-stage. La première image, Node.js, sert uniquement à compiler React. La deuxième image, Nginx, sert uniquement à exécuter le site en production.

Ligne	Explication
<code>FROM node:20-alpine AS build</code>	Demarre une etape de construction nommee <code>build</code> . Node est necessaire pour executer Vite.
<code>WORKDIR /app</code>	Dossier de travail dans l'etape Node.
<code>COPY package.json package-lock.json ./</code>	Copie les fichiers qui definissent les dependances frontend.
<code>RUN npm ci</code>	Installe React, Vite, React Router, Framer Motion et les autres dependances.
<code>COPY index.html vite.config.js styles.css ./</code>	Copie le HTML racine, la configuration Vite et le CSS global.
<code>COPY src/ ./src/</code>	Copie les composants et pages React.
<code>COPY assets/ ./assets/</code>	Copie les images et ressources utilisees par le frontend.
<code>RUN npm run build</code>	Execute <code>vite build</code> . Cela genere le dossier <code>dist</code> .
<code>FROM nginx:alpine</code>	Nouvelle image finale plus legere. Elle ne contient pas Node ni le code source complet.
<code>COPY --from=build /app/dist /usr/share/nginx/html</code>	Copie uniquement le resultat compile dans le dossier servi par Nginx.
<code>COPY nginx.conf /etc/nginx/conf.d/default.conf</code>	Installe la configuration Nginx du projet.
<code>EXPOSE 80</code>	Indique que Nginx ecoute sur le port 80 dans le conteneur.
<code>CMD ["nginx", "-g", "daemon off;"]</code>	Lance Nginx au premier plan. <code>daemon off</code> evite que le conteneur s'arrete immediatement.

8. Le dossier dist

Le dossier `dist` est genere par Vite quand tu executes `npm run build` . Ce n'est pas le code source React. C'est le resultat final optimise pour le navigateur.

Element dans dist	Role
<code>dist/index.html</code>	Page HTML principale envoyee au navigateur.
<code>dist/assets/*.js</code>	JavaScript compile et optimise. Il contient le code React transforme en code executable par le navigateur.
<code>dist/assets/*.css</code>	CSS final minifie.
<code>dist/assets/*image*</code>	Images optimisees et renommees avec un hash pour le cache navigateur.

Dans le Dockerfile frontend, seul `dist` est copié dans l'image finale Nginx. Cela rend l'image finale plus petite et plus propre.

9. Fichier compose.yaml

```
services:
  frontend:
    build: ./reactportfolio
    container_name: portfolio-frontend
    ports:
      - "3000:80"
    depends_on:
      - backend
    networks:
      - portfolio-net

  backend:
    build: "./EXPRESSJS PORTFOLIO"
    container_name: portfolio-backend
    ports:
      - "5000:5000"
    environment:
      - PORT=5000
      - USE_MEMORY_DB=false
      - MONGO_URI=mongodb://mongo:27017/portfolio
    depends_on:
      - mongo
    networks:
      - portfolio-net

  mongo:
    image: mongo:7
    container_name: portfolio-mongo
    ports:
      - "27017:27017"
    volumes:
      - mongo-data:/data/db
    networks:
      - portfolio-net

volumes:
  mongo-data:

networks:
  portfolio-net:
```

Bloc services

`services:` declare les conteneurs que Compose doit créer. Chaque sous-bloc représente un service.

Service frontend

Instruction	Explication
<code>frontend:</code>	Nom logique du service dans Compose.
<code>build: ./reactportfolio</code>	Construit une image a partir du Dockerfile du dossier frontend.
<code>container_name: portfolio-frontend</code>	Nom fixe du conteneur cree.
<code>"3000:80"</code>	Le port 3000 de ta machine redirige vers le port 80 du conteneur Nginx.
<code>depends_on: backend</code>	Compose demarre le backend avant le frontend. Cela ne garantit pas que l'API soit deja prete, seulement que le conteneur est lance.
<code>networks: portfolio-net</code>	Place le frontend sur le reseau prive commun.

Service backend

Instruction	Explication
<code>backend:</code>	Nom logique du service API.
<code>build: "./EXPRESSJS PORTFOLIO"</code>	Construit l'image backend depuis le Dockerfile Express.
<code>container_name: portfolio-backend</code>	Nom fixe du conteneur backend.
<code>"5000:5000"</code>	Expose l'API sur <code>localhost:5000</code> .
<code>PORT=5000</code>	Dit a Express d'ecouter sur le port 5000.
<code>USE_MEMORY_DB=false</code>	Desactive la base en memoire et force l'utilisation de MongoDB.
<code>MONGO_URI=mongodb://mongo:27017/portfolio</code>	Adresse interne de MongoDB dans le reseau Docker.
<code>depends_on: mongo</code>	Demarre MongoDB avant le backend.

Service mongo

Instruction	Explication
<code>mongo:</code>	Nom logique du service base de donnees.
<code>image: mongo:7</code>	Utilise l'image officielle MongoDB version 7 depuis Docker Hub.
<code>container_name: portfolio-mongo</code>	Nom fixe du conteneur MongoDB.
<code>"27017:27017"</code>	Expose MongoDB sur ta machine pour les outils externes.
<code>mongo-data:/data/db</code>	Stocke les fichiers de la base dans un volume persistant.
<code>networks: portfolio-net</code>	Connecte MongoDB au meme reseau que l'API.

10. MongoDB, image Docker Hub et volume

MongoDB n'a pas de Dockerfile local parce que ton projet n'a pas besoin de construire MongoDB. Il utilise une image officielle deja prete :

```
image: mongo:7
```

Si l'image `mongo:7` n'existe pas localement, Docker la recupere automatiquement depuis Docker Hub.

Pourquoi le volume est important

```
volumes:  
- mongo-data:/data/db
```

MongoDB stocke ses donnees dans `/data/db`. Le volume `mongo-data` garde ces donnees en dehors du cycle de vie du conteneur. Tu peux supprimer et recréer le conteneur MongoDB sans perdre les donnees tant que le volume existe.

Commande	Effet
<code>docker compose down</code>	Supprime les conteneurs, mais garde le volume.
<code>docker compose down -v</code>	Supprime les conteneurs et les volumes. Les donnees MongoDB sont supprimees.

11. Configuration Nginx

```
resolver 127.0.0.11 valid=10s;

server {
    listen 80;
    server_name localhost;
    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api {
        set $backend_url http://portfolio-backend:5000;
        proxy_pass $backend_url;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

Ligne	Utilite
<code>resolver 127.0.0.11 valid=10s;</code>	Utilise le DNS interne de Docker. Cela aide Nginx a resoudre les noms de conteneurs sur le reseau Docker.
<code>listen 80;</code>	Nginx ecoute sur le port 80 dans le conteneur.
<code>server_name localhost;</code>	Nom symbolique du serveur Nginx.
<code>root /usr/share/nginx/html;</code>	Dossier ou se trouvent les fichiers React compiles.
<code>index index.html;</code>	Fichier par default servi au navigateur.
<code>location /</code>	Regle pour toutes les pages frontend.
<code>try_files \$uri \$uri/ /index.html;</code>	Si l'URL ne correspond pas a un fichier physique, Nginx renvoie <code>index.html</code> . C'est necessaire pour une SPA React.
<code>location /api</code>	Regle speciale pour les appels API.
<code>set \$backend_url http://portfolio-backend:5000;</code>	Definit l'adresse du backend dans le reseau Docker.
<code>proxy_pass \$backend_url;</code>	Transfere la requete API vers Express.
<code>proxy_http_version 1.1</code>	Utilise HTTP/1.1 pour le proxy.
<code>proxy_set_header Host \$host</code>	Transmet l'hote original au backend.
<code>proxy_cache_bypass \$http_upgrade</code>	Evite certains problemes de cache avec les connexions speciales.

Quand le navigateur appelle `/api/projects` depuis le frontend, Nginx intercepte `/api` et transmet la requete au backend Express.

12. Jenkins avec Docker

docker-compose.jenkins.yml

```
services:
  jenkins:
    build:
      context: .
      dockerfile: Dockerfile.jenkins
    container_name: jenkins-server
    ports:
      - "8080:8080"
      - "50000:50000"
    user: root
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    environment:
      - JAVA_OPTS=-Djenkins.install.runSetupWizard=true
    restart: unless-stopped

volumes:
  jenkins_home:
```

Instruction	Explication
<code>jenkins:</code>	Service Jenkins.
<code>build: context: .</code>	Construit l'image Jenkins depuis le dossier courant.
<code>dockerfile: Dockerfile.jenkins</code>	Indique le Dockerfile special Jenkins.
<code>container_name: jenkins-server</code>	Nom fixe du conteneur Jenkins.
<code>"8080:8080"</code>	Expose l'interface Jenkins sur <code>http://localhost:8080</code> .
<code>"50000:50000"</code>	Port utilise pour les agents Jenkins.
<code>user: root</code>	Permet a Jenkins d'avoir les droits necessaires pour utiliser Docker dans ce contexte.
<code>jenkins_home:/var/jenkins_home</code>	Volume persistant pour garder la configuration Jenkins, les jobs et plugins.
<code>/var/run/docker.sock:/var/run/docker.sock</code>	Donne au Jenkins conteneurise l'accès au Docker de la machine hôte.
<code>JAVA_OPTS=...</code>	Configure Jenkins pour afficher l'assistant d'installation au démarrage.
<code>restart: unless-stopped</code>	Relance Jenkins automatiquement sauf si tu l'arretes volontairement.

Dockerfile.jenkins

```
FROM jenkins/jenkins:lts

USER root

# Install Node.js 20
RUN apt-get update && apt-get install -y curl && \
    curl -fsSL https://deb.nodesource.com/setup_20.x | bash - && \
    apt-get install -y nodejs

# Install Docker CLI
RUN apt-get install -y docker.io

# Autoriser Git dans tous les repertoires (necessaire quand root)
RUN git config --global --add safe.directory '*'
```

Ligne	Explication
<code>FROM jenkins/jenkins:lts</code>	Utilise l'image officielle Jenkins Long Term Support.
<code>USER root</code>	Passe en root pour installer des paquets systeme.
<code>apt-get update</code>	Met a jour la liste des paquets Debian.
<code>apt-get install -y curl</code>	Installe curl pour telecharger le script NodeSource.
<code>curl -fsSL ...setup_20.x bash -</code>	Ajoute le depot Node.js 20.
<code>apt-get install -y nodejs</code>	Installe Node.js et npm dans le conteneur Jenkins.
<code>apt-get install -y docker.io</code>	Installe la CLI Docker pour que Jenkins puisse executer <code>docker build</code> et <code>docker run</code> .
<code>git config --global --add safe.directory '*'</code>	Evite les erreurs Git de securite quand Jenkins travaille en root.

13. Jenkinsfile ligne par ligne

Le `Jenkinsfile` décrit le pipeline CI/CD. Jenkins lit ce fichier depuis GitHub, puis exécute les étapes dans l'ordre.

Structure globale

```
pipeline {  
  agent any  
  environment { ... }  
  triggers { ... }  
  stages { ... }  
  post { ... }  
}
```

Element	Utilite
<code>pipeline {}</code>	Declare un pipeline Jenkins declaratif.
<code>agent any</code>	Jenkins peut executer ce pipeline sur n'importe quel agent disponible.
<code>environment {}</code>	Definit des variables globales reutilisables dans les etapes.
<code>triggers {}</code>	Definit ce qui declenche automatiquement le pipeline.
<code>stages {}</code>	Contient les grandes etapes : checkout, install, tests, build, deploy.
<code>post {}</code>	Actions executees apres le pipeline : succes, echec, nettoyage.

Variables d'environnement Jenkins

```
environment {  
  FRONTEND_DIR = 'REACTPORTFOLIO/reactportfolio'  
  BACKEND_DIR = 'REACTPORTFOLIO/EXPRESSJS PORTFOLIO'  
  EMAIL_RECIPIENTS = 'mouhamed.sall@email.com'  
}
```

Variable	Utilite
<code>FRONTEND_DIR</code>	Chemin du frontend dans le workspace Jenkins.
<code>BACKEND_DIR</code>	Chemin du backend. Les guillemets sont importants car le dossier contient un espace.
<code>EMAIL_RECIPIENTS</code>	Adresse qui recevra les notifications de succes ou d'echec.

Declencheur GitHub

```
triggers {  
  githubPush()  
}
```

Cette instruction indique que Jenkins peut lancer le pipeline quand GitHub envoie un evenement de push, si le webhook GitHub est configure.

Stage Checkout

```
stage('Checkout') {  
  steps {  
    echo 'Recuperation du code source...'  
    checkout scm  
  }  
}
```

Element	Utilite
<code>stage('Checkout')</code>	Nom visible dans Jenkins pour l'etape de recuperation du code.
<code>steps {}</code>	Liste les actions de cette etape.
<code>echo</code>	Affiche un message dans les logs Jenkins.
<code>checkout scm</code>	Recupere le code depuis le depot Git configure dans le job Jenkins.

Stage Install Dependencies

```
stage('Install Dependencies') {  
  parallel {  
    stage('Frontend Dependencies') {  
      steps {  
        dir("${env.FRONTEND_DIR}") {  
          sh 'npm install'  
        }  
      }  
    }  
    stage('Backend Dependencies') {  
      steps {  
        dir("${env.BACKEND_DIR}") {  
          sh 'npm install'  
        }  
      }  
    }  
  }  
}
```

Cette etape installe les dependances frontend et backend en parallele. Cela accelere le pipeline car les deux installations ne dependent pas l'une de l'autre.

Element	Utilite
<code>parallel</code>	Lance plusieurs branches en meme temps.
<code>dir("\${env.FRONTEND_DIR}")</code>	Entre dans le dossier frontend avant d'executer la commande.
<code>dir("\${env.BACKEND_DIR}")</code>	Entre dans le dossier backend avant d'executer la commande.
<code>sh 'npm install'</code>	Installe les dependances dans le workspace Jenkins. Pour un build plus reproductible, <code>npm ci</code> peut aussi etre utilise ici.

Stage Tests

```
stage('Tests') {  
  steps {  
    dir("${env.BACKEND_DIR}") {  
      sh 'npm test'  
    }  
  }  
}
```

Jenkins execute les tests backend. Dans ton `package.json`, `npm test` lance `node --test`.

Stage Build Frontend

```
stage('Build Frontend') {  
  steps {  
    dir("${env.FRONTEND_DIR}") {  
      sh 'npm run build'  
    }  
  }  
}
```

Cette etape compile React avec Vite. Elle genere un dossier `dist` dans le workspace Jenkins. Ensuite, le Dockerfile frontend reconstruit aussi le frontend dans l'image Docker pour produire une image autonome.

Stage Docker Build

```
stage('Docker Build') {  
  steps {  
    dir('REACTPORTFOLIO') {  
      sh 'docker build -t portfolio-frontend ./reactportfolio'  
      sh 'docker build -t portfolio-backend "./EXPRESSJS PORTFOLIO"  
    }  
  }  
}
```

Commande	Utilite
<code>dir('REACTPORTFOLIO')</code>	Se place a la racine du projet clone dans Jenkins.
<code>docker build -t portfolio-frontend ./reactportfolio</code>	Construit l'image frontend et lui donne le tag <code>portfolio-frontend</code> .
<code>docker build -t portfolio-backend "./EXPRESSJS PORTFOLIO"</code>	Construit l'image backend. Les guillemets protegent l'espace dans le nom du dossier.

Stage Deploy

```
stage('Deploy') {  
  steps {  
    sh '''  
      docker stop portfolio-frontend portfolio-backend portfolio-mongo || true  
      docker rm portfolio-frontend portfolio-backend portfolio-mongo || true  
      docker network create portfolio-net || true  
      docker run -d --name portfolio-mongo --network portfolio-net --network-alias mongo mongo:7  
      sleep 5  
      docker run -d --name portfolio-backend --network portfolio-net --network-alias backend -p  
5000:5000 -e PORT=5000 -e MONGO_URI=mongodb://mongo:27017/portfolio -e USE_MEMORY_DB=false portfolio-  
backend  
      sleep 3  
      docker run -d --name portfolio-frontend --network portfolio-net --network-alias frontend -p  
3000:80 portfolio-frontend  
      '''  
    }  
  }  
}
```

Commande	Explication
<code>docker stop ... true</code>	Arrete les anciens conteneurs s'ils existent. <code> true</code> evite que Jenkins echoue si le conteneur n'existe pas.
<code>docker rm ... true</code>	Supprime les anciens conteneurs pour pouvoir recreeer des conteneurs avec les memes noms.
<code>docker network create portfolio-net true</code>	Cree le reseau Docker. Si le reseau existe deja, l'erreur est ignoree.
<code>docker run -d --name portfolio-mongo ... mongo:7</code>	Lance MongoDB en arriere-plan depuis l'image officielle.
<code>--network portfolio-net</code>	Connecte le conteneur au reseau commun.
<code>--network-alias mongo</code>	Permet au backend d'utiliser le nom <code>mongo</code> dans <code>MONGO_URI</code> .
<code>sleep 5</code>	Laisse quelques secondes a MongoDB pour demarrer.
<code>docker run -d --name portfolio-backend ...</code>	Lance l'API Express.
<code>-p 5000:5000</code>	Publie l'API sur le port 5000 de la machine.
<code>-e PORT=5000</code>	Injecte la variable d'environnement du port Express.
<code>-e MONGO_URI=mongodb://mongo:27017/portfolio</code>	Indique au backend l'adresse interne de MongoDB.
<code>-e USE_MEMORY_DB=false</code>	Force le backend a utiliser MongoDB reel.
<code>--network-alias backend</code>	Permet d'appeler le backend avec le nom <code>backend</code> si necessaire.
<code>sleep 3</code>	Laisse quelques secondes au backend pour demarrer.
<code>docker run -d --name portfolio-frontend ...</code>	Lance Nginx avec le frontend React compile.
<code>-p 3000:80</code>	Publie le frontend sur <code>http://localhost:3000</code> .

Stage Health Check

```
stage('Health Check') {
  steps {
    sh '''
      sleep 10
      docker exec portfolio-frontend wget -qO- http://127.0.0.1:80 > /dev/null || exit 1
      docker exec portfolio-backend node -e "fetch('http://127.0.0.1:5000/api/projects').then(r=>
      {if(!r.ok)process.exit(1);process.exit(0)}).catch(()=>process.exit(1))"
    '''
  }
}
```

Element	Utilite
<code>sleep 10</code>	Attend que les services soient prêts.
<code>docker exec portfolio-frontend ...</code>	Execute une commande a l'interieur du conteneur frontend.
<code>wget -q0-http://127.0.0.1:80</code>	Teste que Nginx repond depuis l'interieur du conteneur.
<code>> /dev/null</code>	Cache le HTML reçu pour garder les logs propres.
<code> exit 1</code>	Fait echouer le pipeline si le frontend ne repond pas.
<code>node -e "fetch(...)"</code>	Execute un mini-script Node pour tester l'API backend.
<code>127.0.0.1</code>	Force l'adresse IPv4 locale dans le conteneur. Cela evite certains problemes de resolution de <code>localhost</code> .

Bloc post

```
post {
  success { ... mail ... }
  failure { ... stop/rm ... mail ... }
  always { cleanWs() }
}
```

Bloc	Explication
<code>success</code>	Execute si tout le pipeline reussit. Envoie un email de succes.
<code>failure</code>	Execute si une etape echoue. Arrete et supprime les conteneurs, puis envoie un email d'echec.
<code>always</code>	Execute quoi qu'il arrive. Nettoie le workspace Jenkins avec <code>cleanWs()</code> .

14. Health checks et correction de l'erreur du pipeline

L'erreur vue dans les logs etait :

```
Error response from daemon: container ... is not running
```

Cela signifie que Jenkins avait cree le conteneur frontend, mais que celui-ci s'etait arrete avant la verification.

Pourquoi un conteneur peut s'arreter ?

Un conteneur reste vivant tant que son processus principal reste vivant. Pour le frontend, le processus principal est Nginx :

```
CMD ["nginx", "-g", "daemon off;"]
```

Si Nginx ne parvient pas à charger correctement sa configuration ou à résoudre son upstream backend, il peut quitter, et le conteneur s'arrête.

Les corrections importantes

Correction	Pourquoi
<code>--network-alias mongo</code>	Permet au backend de trouver MongoDB avec le nom <code>mongo</code> .
<code>--network-alias backend</code>	Permet aux autres conteneurs d'utiliser le nom <code>backend</code> .
<code>resolver 127.0.0.11</code> dans Nginx	Utilise le DNS interne Docker.
<code>127.0.0.1</code> au lieu de <code>localhost</code> dans les health checks	Évite les problèmes de résolution IPv6/IPv4 dans certains conteneurs.
<code>sleep 5</code> , <code>sleep 3</code> , <code>sleep 10</code>	Laisse aux services le temps de démarrer avant les tests.

Important : les messages `No such container` au début du deploy ne sont pas forcément graves, car les commandes ont `|| true`. Jenkins les ignore volontairement pour permettre le premier déploiement.

15. Commandes utiles

Docker Compose

```
docker compose up --build
docker compose up --build -d
docker compose down
docker compose down -v
```

Images

```
docker images
docker build -t portfolio-frontend ./reactportfolio
docker build -t portfolio-backend "./EXPRESSJS PORTFOLIO"
```

Conteneurs

```
docker ps
docker ps -a
docker logs portfolio-backend
docker logs portfolio-frontend
docker exec -it portfolio-backend sh
```

Reseaux et volumes

```
docker network ls
docker network inspect portfolio-net
docker volume ls
docker volume inspect reactportfolio_mongo-data
```

Tests rapides

```
curl http://localhost:3000
curl http://localhost:5000/api/projects
docker exec portfolio-frontend wget -q0- http://127.0.0.1:80
docker exec portfolio-backend node -e
"fetch('http://127.0.0.1:5000/api/projects').then(r=>console.log(r.status))"
```

16. Resume final

Dockerfile backend

- > construit l'image portfolio-backend
- > lance Express avec node app.js
- > Express ecoute sur 5000
- > Express se connecte a mongo:27017

Dockerfile frontend

- > construit React avec Node
- > genere dist
- > copie dist dans Nginx
- > Nginx ecoute sur 80
- > machine expose le site sur localhost:3000

compose.yaml

- > construit frontend et backend
- > telecharge mongo:7
- > cree les conteneurs
- > cree le reseau portfolio-net
- > cree le volume mongo-data

Jenkinsfile

- > recupere le code
- > installe les dependances
- > lance les tests
- > build le frontend
- > construit les images Docker
- > deploie les conteneurs
- > verifie frontend et backend
- > nettoie et envoie un email